Project Proposal Template

Project: E-Commerce Data Integration & Analytics Pipeline

Objective: To build an end-to-end e-commerce data integration platform that collects the prices of the same products from three Egyptian online retailers, matches them into one product list, combines them with the sales of our own store, and loads both incrementally into a PostgreSQL star-schema data warehouse with Python ETL orchestrated by Apache Airflow, reported with SQL analytics and a Power BI dashboard.

Description: An electronics retailer wants to know every day how its prices compare with the market and how that affects its sales. The same phone or TV is sold on several websites, each with its own page format, product names and model codes, so the data cannot be compared until it is integrated. This project scrapes three Egyptian retailers once a day: XPRS and Dream 2000 (public Shopify JSON) and Jumia Egypt (HTML pages; its robots.txt permits identified bots), politely (one request every 1 to 5 seconds). The raw offers are stored in MongoDB and matched into one product list by brand and normalised model code, so adding a new site adds rows, not tables. Sales come from the operational MySQL database of our own store, filled with generated (synthetic) customers and orders for the same real products in daily batches, as approved by the instructor. All amounts are in Egyptian pounds (EGP). The five modules follow the structure of the reference repository.

Tools & Design: The platform has five modules, each delivered as a numbered folder in the GitHub repository with its scripts, data samples, screenshots and a Markdown guide, as shown below:

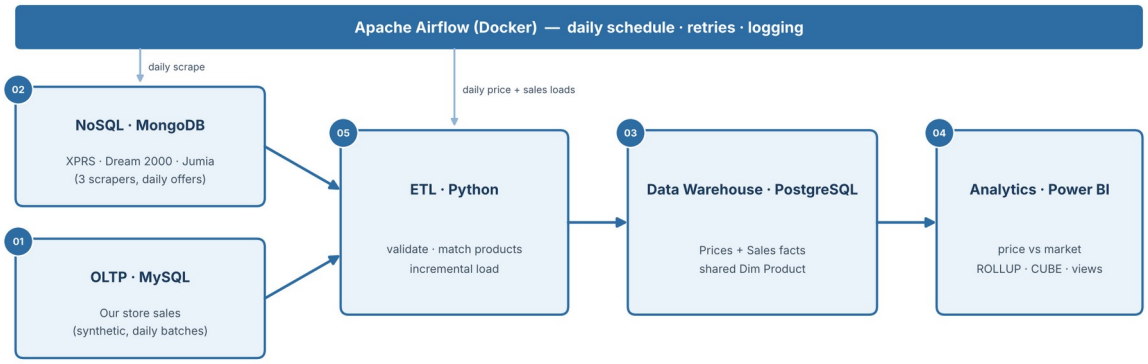


Figure 1. Architecture: three websites scraped into MongoDB and synthetic sales in MySQL, integrated by Python ETL (product matching and incremental loads) orchestrated by Apache Airflow into a PostgreSQL warehouse with two fact tables sharing one product dimension, and a Power BI dashboard.

- 01 OLTP (MySQL) — our store's customers, products, orders and order items (synthetic, generated as daily CSV files) in a normalised schema; index on the purchase timestamp; export script with mysqldump.
- 02 NoSQL (MongoDB) — daily offers from XPRS, Dream 2000 and Jumia stored as documents in each site's own structure; indexes, queries and export to CSV.
- 03 Data Warehouse (PostgreSQL) — star schema with two fact tables (Fact Price Daily, Fact Sales) sharing Dim Product and Dim Date, plus Dim Site, Dim Customer and an offer-to-product map.
- 04 Analytics & Dashboard — GROUPING SETS, ROLLUP, CUBE, views and a materialized view in SQL for six reports (cheapest site, price spread, discounts, stock, our sales, our price against the market), and a Power BI dashboard.
- 05 ETL & Pipelines — product matching (brand and normalised model code), daily price load and incremental sales load with data validation and quality checks, run by two Apache Airflow DAGs with schedules, retries and logging.

Technologies to Use: The following technologies are used across the platform:

- Python (pandas, requests, BeautifulSoup, mysql-connector-python, psycopg2, pymongo) — scrapers, sales generator and ETL scripts
- MySQL — operational (OLTP) sales database of our store
- MongoDB — NoSQL store of the raw site offers
- PostgreSQL — data warehouse (star schema)
- Apache Airflow — orchestration, scheduling, retries and logging
- Docker Compose — runs Airflow and the three databases
- Power BI Desktop — business-intelligence dashboard
- DBeaver — database management and querying (MySQL and PostgreSQL)
- Bash and Git / GitHub — export scripts, version control and project repository

---

Team Members & Responsibilities:

1- Mahmoud Dahroug — Team Leader & BI — Coordination, data dictionary, proposal and README, weekly review of product matches, Power BI dashboard, final presentation

2- Aya Elgarwany — Data Extraction & NoSQL — Overlap test, scrapers for XPRS, Dream 2000 and Jumia, daily scraping, MongoDB offers collection, queries and export

3- Hussein Saeed — Database Engineer — MySQL OLTP schema, synthetic sales generator and loads, PostgreSQL star schema and ERD, analytical SQL

4- Nouran Elshwehy — ETL & Orchestration — Docker setup, product matching, price and sales ETL, Airflow DAGs, failure tests

5- Islam Hassan — Documentation & Quality Support — Second check of product matches, weekly tracker follow-up, presentation draft, proofreading

---

Week 1: Redesign, Environment & Operational Database

- Mahmoud — Close the open design decisions in the data dictionary (v4.1) and share it with the team. (M1)
- Mahmoud — Create the new GitHub repository ecommerce-data-pipeline with the module guides, code skeletons and reused infrastructure, and add the team. (M2)
- Hussein — Write oltp.sql for our store (customers, products, orders, order items) with one character set and collation for all tables, and push it to GitHub. (H1)
- Nouran — Run Docker Compose on the team environment and prepare MongoDB and PostgreSQL for the new design. (N1)

Deliverables:

- Data dictionary v4.3 with the matching rules
- New GitHub repository (ecommerce-data-pipeline)
- MySQL OLTP schema (oltp.sql)
- Docker Compose environment with Airflow and the three databases

---

Week 2: Data Collection, Warehouse Schema & Product Matching

- Mahmoud — Update the proposal, README and architecture diagram to the new design. (M3)
- Aya — Run the overlap test on Dream 2000, Jumia and XPRS: phones and TVs kept, air conditioners dropped (fewer than 20 products on two or more sites). (A1)
- Aya — Write the Dream 2000 scraper (public products.json). (A2)

- Aya — Adapt mongo_load.py to the catalog.offers collection with indexes and a quarantine collection. (A3)
- Aya — Run the scraping every day from 27 October (by hand until the DAG is ready), so the price history starts early. (A4)
- Aya — Write the XPRS scraper (public products.json, same code as Dream 2000; replaces 2B, which blocks bots). (A5)
- Aya — Write the Jumia scraper (HTML pages; User-Agent with the bot name and a contact address). (A6)
- Aya — Build the brand alias list (Arabic and English) and the model-code suffix list from the first full scrape. (A7)
- Hussein — Write schema.sql for the warehouse (seven tables, Dim Date and the etl_state control table) and draw the ERD. (H2)
- Hussein — Create the warehouse tables on PostgreSQL in the team environment. (H3)
- Hussein — Write generate_sales.py, part 1: the customers of our store. (H4)
- Hussein — Write generate_sales.py, part 2: our products and daily orders, the batch load script and datadump.sh. (H5)
- Nouran — Write the product-matching step: normalise brands and model codes and build Dim Product and the offer-to-product map. (N2)
- Islam — Follow up the tracker every Thursday and send the team a short note of late tasks. (I2)

Deliverables:

- Updated proposal, README and architecture diagram
- Overlap test results
- Three scrapers and daily snapshots in MongoDB
- Brand alias and model-code suffix lists
- Warehouse schema (schema.sql) and ERD
- Sales generator, load script and datadump.sh
- Product-matching step

Week 3: Price & Sales ETL, Orchestration & Analytics

- Mahmoud — Review 20 random product matches every week and record corrections in manual_matches.csv. (M4)
- Islam — Second check of the same 20 matches on the websites, with notes. (I1)
- Mahmoud — Install Power BI Desktop and connect it to the PostgreSQL warehouse. (M5)
- Mahmoud — Build the Power BI dashboard: six report pages. (M6)

- Aya — Write the MongoDB queries and export the offers to CSV with mongoexport. (A8)
- Hussein — Write the analytical SQL: views, a materialized view, GROUPING SETS, ROLLUP and CUBE for the six reports. (H6)
- Nouran — Build the price ETL from MongoDB to Fact Price Daily and load every day collected so far. (N3)
- Nouran — Adapt the incremental sales ETL (automation.py) to the new tables. (N4)
- Nouran — Build the price DAG: scrape → MongoDB → validate → match → load → quality check. (N5)
- Nouran — Build the sales DAG: extract → validate → transform → load → quality check. (N6)

Deliverables:

- Fact Price Daily and Fact Sales loaded
- Two Airflow DAGs with successful runs
- MongoDB queries and CSV export
- Analytical SQL scripts and materialized view
- Power BI dashboard (.pbix file and screenshots)

Week 4: Testing, Documentation & Final Presentation

- Mahmoud — Coordinate an end-to-end run, finalise the README and the presentation, and lead the demo dry run. (M7)
- Aya — Write the Module 02 guide with steps, commands and screenshots. (A9)
- Hussein & Nouran — Run failure tests (website unreachable, bad batch, database down) to show retries and error handling, and fix issues. (H7, N7)
- Hussein — Write the Module 01 and 03 guides with steps, commands and screenshots. (H8)
- Nouran — Write the Module 05 guide with the DAG, log and etl_state screenshots. (N8)
- Islam — Prepare the first draft of the presentation slides from the proposal and README. (I4)
- Islam — Proofread the module guides and the README, and check every link and screenshot. (I3)

Deliverables:

- Complete GitHub repository following the reference structure
- README with architecture diagram

- Module guides 01–05 (Markdown with screenshots)
- Failure-test log
- Final presentation and live demo